Technische Hochschule Ingolstadt

Fakultät für Elektrotechnik und Informatik

Zukunft in Bewegung

Serverpatterns

Dokumenthistorie

Autor: Prof. Dr. Bernd Hafenrichter

Version: 1.3

Letze Änderung: 14.04.2026

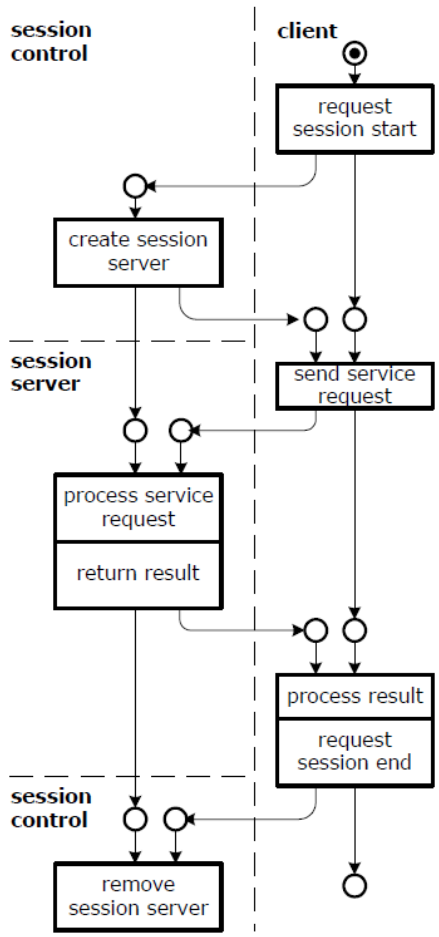
Konzeptionelle Serverpatterns

Motivation:

- Moderne Softwaresysteme sind unterteilt in einen Client- und einen Serverteil
- Die Serverkomponenten sind in der Regel Server welche auf Multitasking aufsetzen um eingehende Aufträge zu bearbeiten
- Nachfolgend wird ein Liste von Patterns für auftragsbasierte Server vorgestellt.

- Achtung: Die vorgestellten Patterns können auch auf andere Aufgabenstellungen übertragen werden. Sie können dann angewendet werden, wenn ein definierter Umgang mit Ressourcen notwendig ist

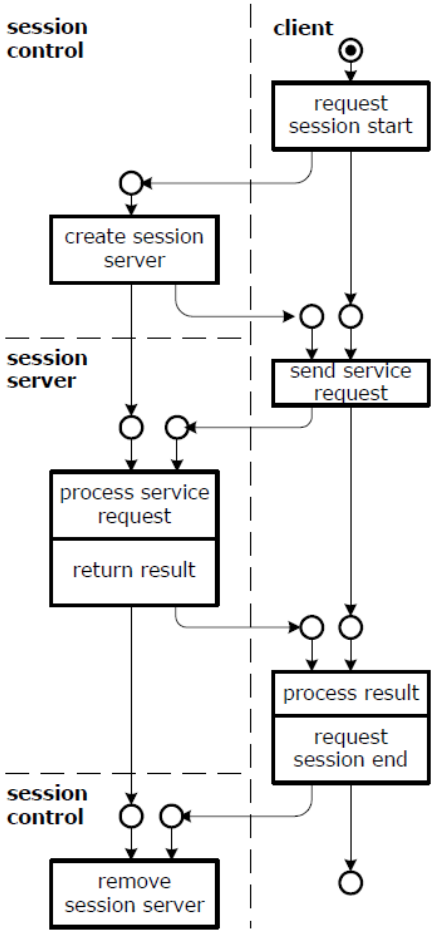
Konzeptionelle Serverpatterns



Single-Request-Session:

- Der Client baut eine Sitzung mit dem Server auf
- Der Client sendet den Auftrag an den Server
- Der Server verarbeitet den Auftrag
- Der Server sendet die Antwort zurück an den Client
- Die Verbindung zwischen Client uns Server wird wieder abgebaut

Konzeptionelle Serverpatterns



Folgende Zeiten sind relevant für den Client:

- Connect Time: Die Dauer zwischen Senden einer (TCP-) Verbindungsanfrage und Aufbau der Verbindung
- Response Time: Zeit für die Antwort auf den Auftrag nach Verbindungsaufbau

Konzeptionelle Serverpatterns – Sequentieller Server

Pattern: Sequentieller Server

Kontext :

Ein Server soll eingehende Client-Anfragen verarbeiten.

Konzeptionelle Serverpatterns – Sequentieller Server

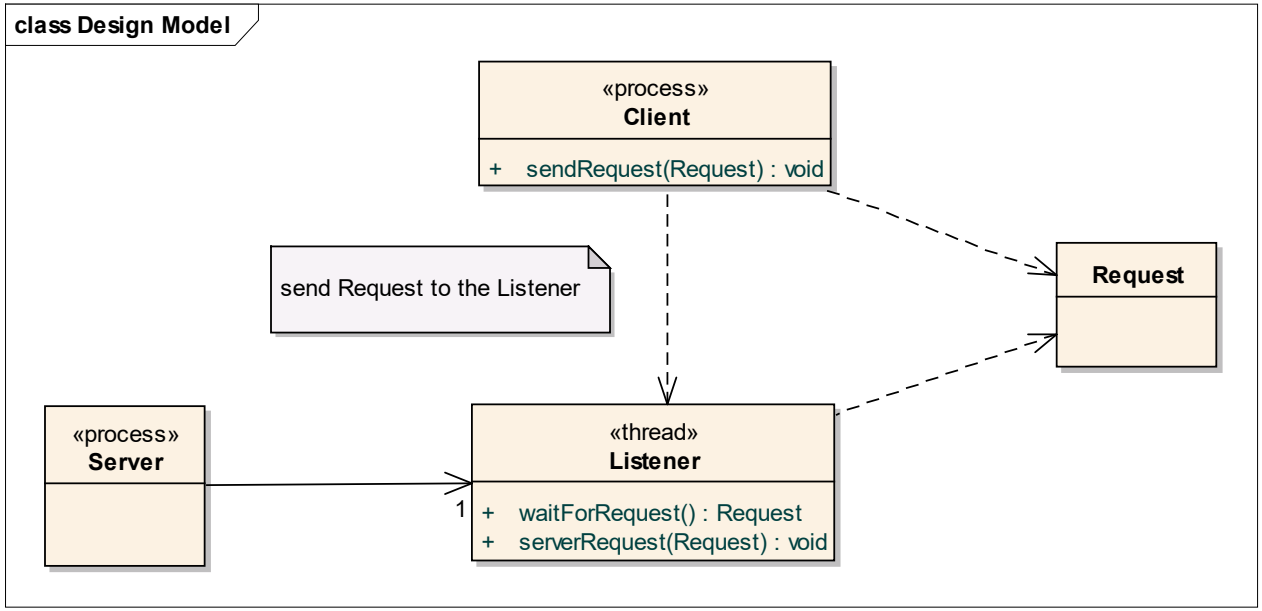
Lösung: Ein Server-Thread ist für die Abarbeitung der Anfragen zuständig

Server-Thread:

- Ein Thread überwacht den oder die Server Ports
- Bei einem Connection-Request wird die Verbindung zum Client aufgebaut.
- Danach wird die Anfrage verarbeitet und die Verbindung geschlossen
- Danach können neue Anfragen bearbeitet werden

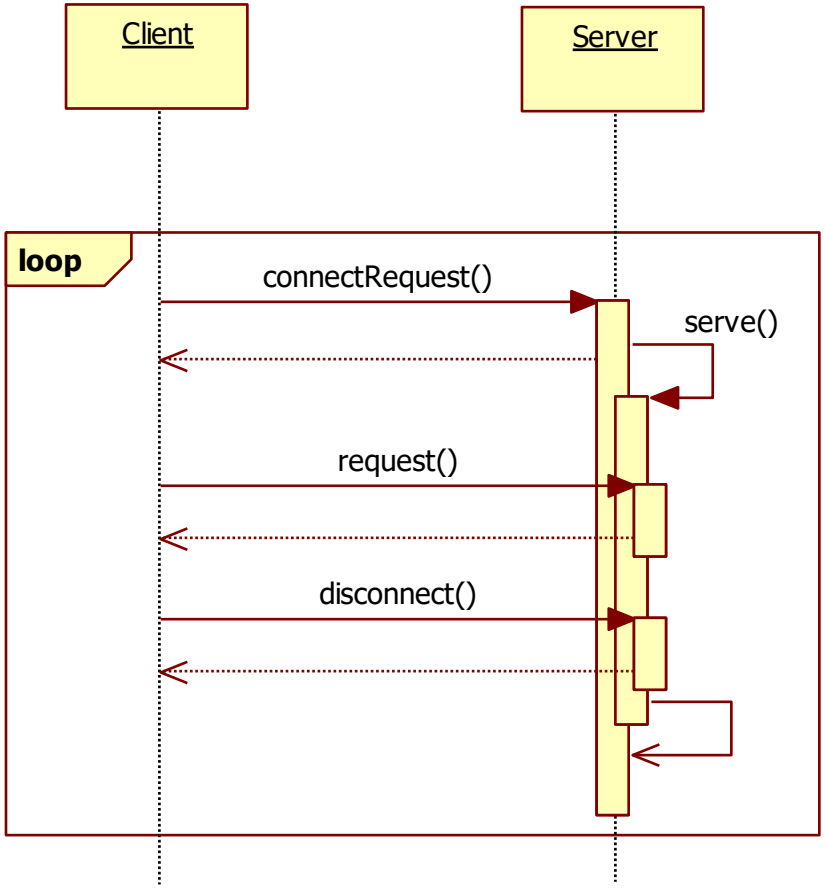
Konzeptionelle Serverpatterns - Sequentieller Server

Statische Sichtweise



Konzeptionelle Serverpatterns - Sequentieller Server

Dynamische Sichtweise



Konzeptionelle Serverpatterns – Sequentieller Server

Eigenschaften:

- Die Implementierung dieses Musters ist sehr einfach
- Es erfolgt eine Zwangsserialisierung der eingehenden Anfragen
- Eine Synchronisation ist nicht notwendig
- Ressourcennutzung ist exakt definiert

Problem:

- Die vorhandenen Maschinenresourcen werden nicht optimal ausgenutzt

Konzeptionelle Serverpatterns – Listener Worker

Pattern: Listener/Worker

Kontext :

Ein Server soll mit Hilfe von Multitasking Dienste für eine offene Anzahl an Clients nebenläufig anbieten. Es wird ein verbindungsorientiertes Netzwerkprotokoll (z.B. TCP/IP) verwendet.

Problem

Wie benutzt man Tasks (Prozesse oder Threads) zur nebenläufigen Bearbeitung von Connection– und Service–Requests?

Konzeptionelle Serverpatterns - Listener Worker

Lösung: Unterteile die Tasks im Server in Listener und Worker Tasks:

Listener:

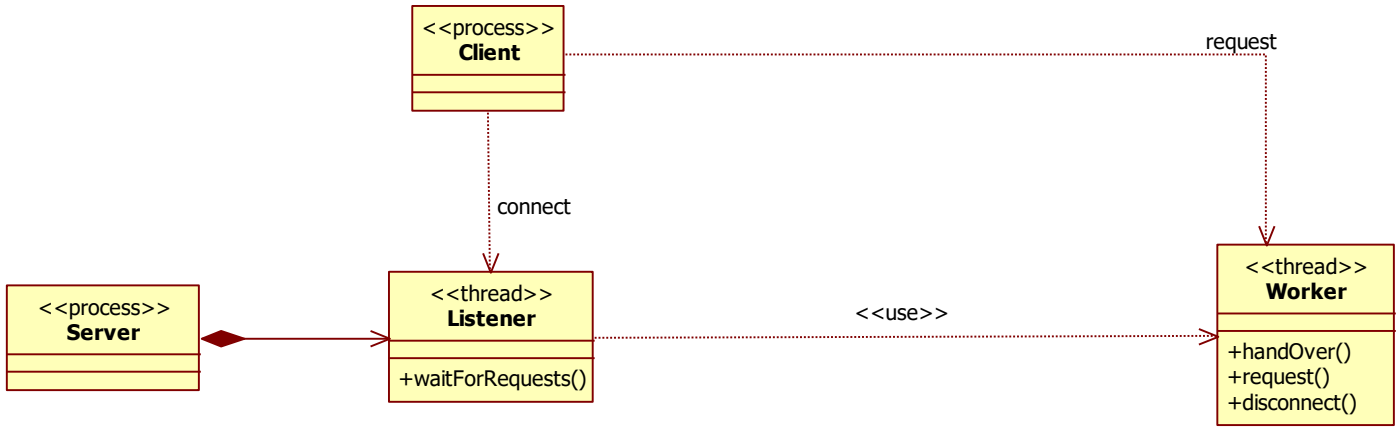
Eine Task überwacht den oder die Server Ports, baut bei einem Connection Request die Verbindung zum Client auf und übergibt diese Verbindung an einen Worker.

Worker:

Der Worker verarbeit eingehende Aufträge des Clients und liefert das gewünschte Ergebnis zurück. Idealerweise gibt es pro Client ein Worker Task, welcher ausschließlich für den Client aktiv ist.

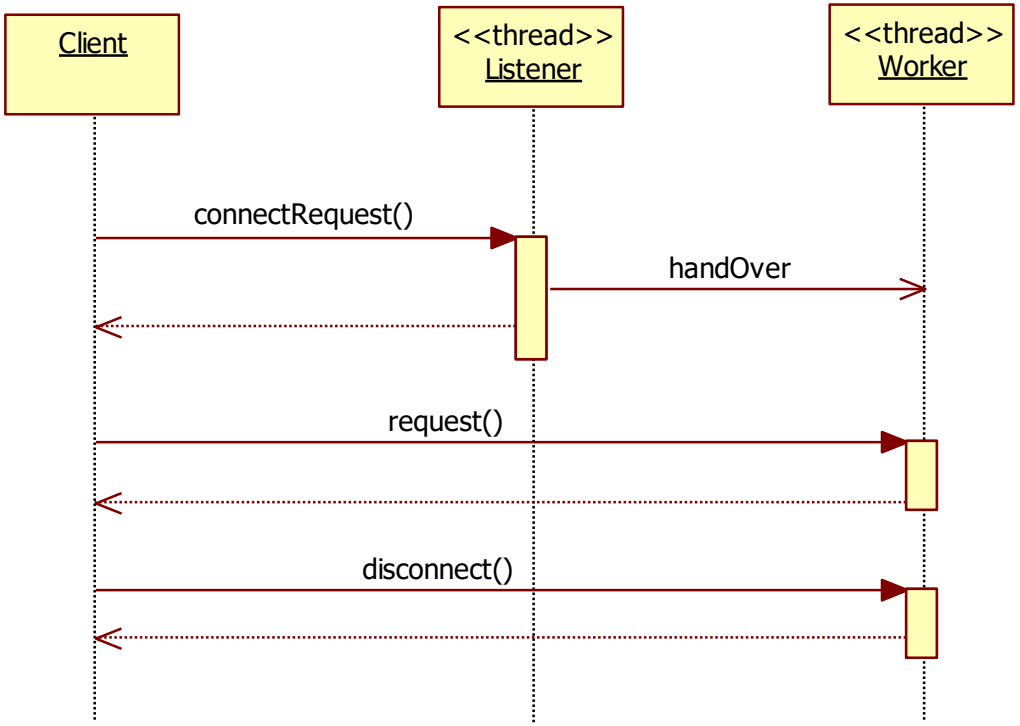
Konzeptionelle Serverpatterns - Listener Worker

Statische Sichtweise



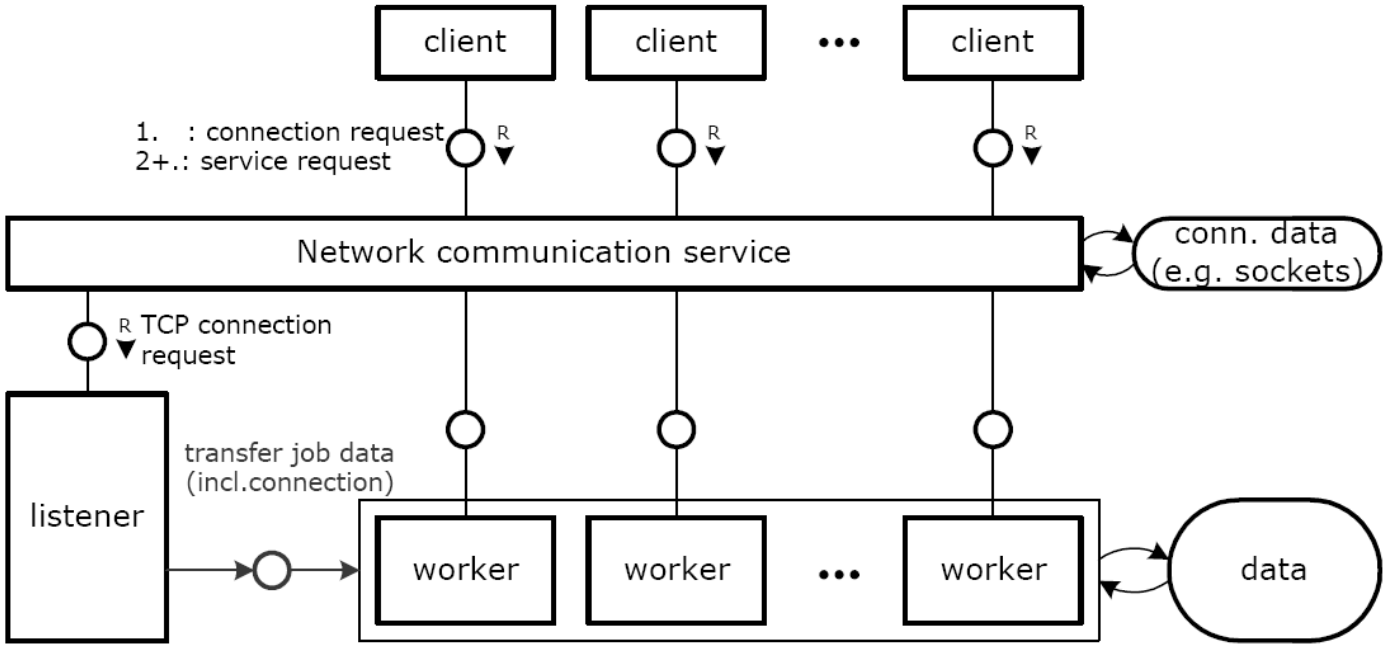
Konzeptionelle Serverpatterns - Listener Worker

Dynamische Sichtweise



Konzeptionelle Serverpatterns - Listener Worker

Statische Sichtweise



Konzeptionelle Serverpatterns - Listener Worker

Eigenschaften:

- Der Verbindungsaufbau erfolgt schnell, da der Listener nur die Aufgabe hat, auf Connection Requests zu reagieren.
- Synchronisation beim Zugriff auf Resourcen notwendig

Offene Fragen/Punkte

- Wann werden die Worker Tasks erzeugt?
- Wie übergibt der Listener die Verbindung zum Client an einen Worker?

Konzeptionelle Serverpatterns – Forking Server

Pattern: Forking Server

Ausgangssituation

Man wendet für einen auftragsbearbeitenden Server das LISTENER / WORKER Pattern an.

Problem

Wie handhabt man möglichst einfach die Erzeugung der Worker Tasks und die Übergabe der Client–Verbindung?

Konzeptionelle Serverpatterns – Forking Server

Lösung: Für jeden Connection-Request wird ein eigenen Worker gestartet.

- Starte eine Task als Master Server, die die Rolle des Listeners übernimmt
- Für jeden eingehenden Connection-Request wird ein neuer Worker-Task erzeugt
    - Unix: fork
    - Windows: CreateProcess
    - Alternativ: Thread
- Der Worker-Task bearbeitet die Service-Request und beendet sich nach getaner Arbeit

Konzeptionelle Serverpatterns – Forking Server

Der Namensgeber – fork()

- Der Systemaufruf „fork()“ dient dazu, unter Linux und Unix-Systemen einen neuen Prozess zu erstellen
- Der neu erzeugte Prozess wird als „Kindprozess“ bezeichnet du läuft parallel zu dem erzeugenden Prozess (Elternprozess läuft)
- Nachdem ein neuer Kindprozess erstellt wurde, führen beide Prozesse die nächste Anweisung nach dem Systemaufruf „fork()“ aus.

```
pid_t p = fork(); // Create a new process as „copy“ of the current one
```

Die Rückgabewerte von fork()

- Negativer Wert: Die Erstellung eines Kindprozesses ist fehlgeschlagen; in diesem Fall wird -1 zurückgegeben.
- 0: Wird an den neu erstellten Kindprozess zurückgegeben.
- Positiver Wert: Wird an den Elternprozess oder den Aufrufer zurückgegeben. Der Wert enthält die Prozess-ID des neu erstellten Kindprozesses.

https://www.geeksforgeeks.org/c/fork-system-call/

Konzeptionelle Serverpatterns – Forking Server

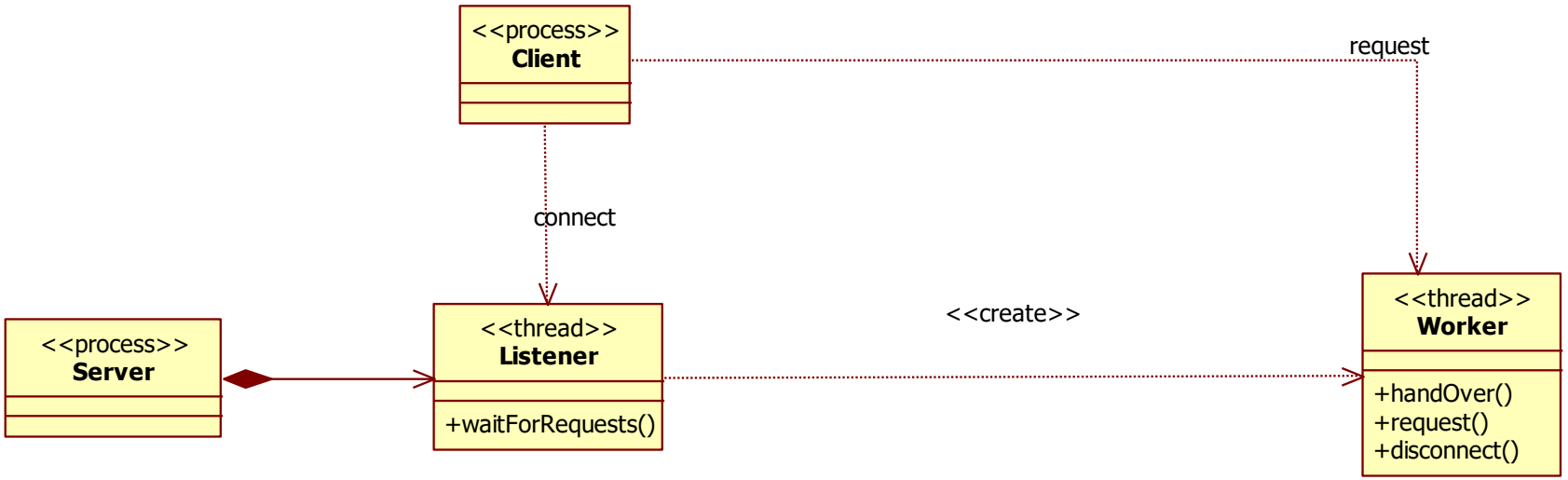
Der Namensgeber – fork()

```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>

int main() {

    pid_t p = fork(); // Create a second process as „copy“ of the current one
    if(p<0){
      perror("fork fail");
      exit(1);
    }
    else if( p == 0 ) {
         printf("Child process(pid) = %d started \n",getpid());
    }
     else {
         printf("Parent process(pid) = %d started child(%d)\n",getpid(),p);
    }

    printf("Hello world!, process_id(pid) = %d \n",getpid());
    return 0;
}
```

https://www.geeksforgeeks.org/c/fork-system-call/

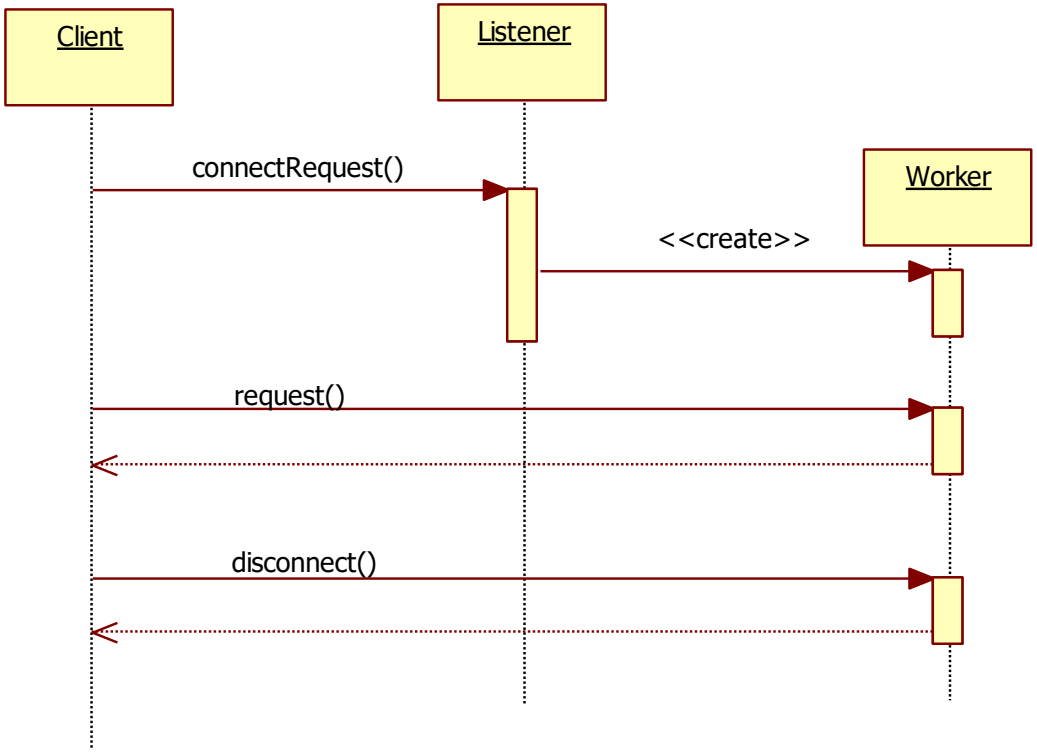
Konzeptionelle Serverpatterns – Forking Server

Statische Sichtweise



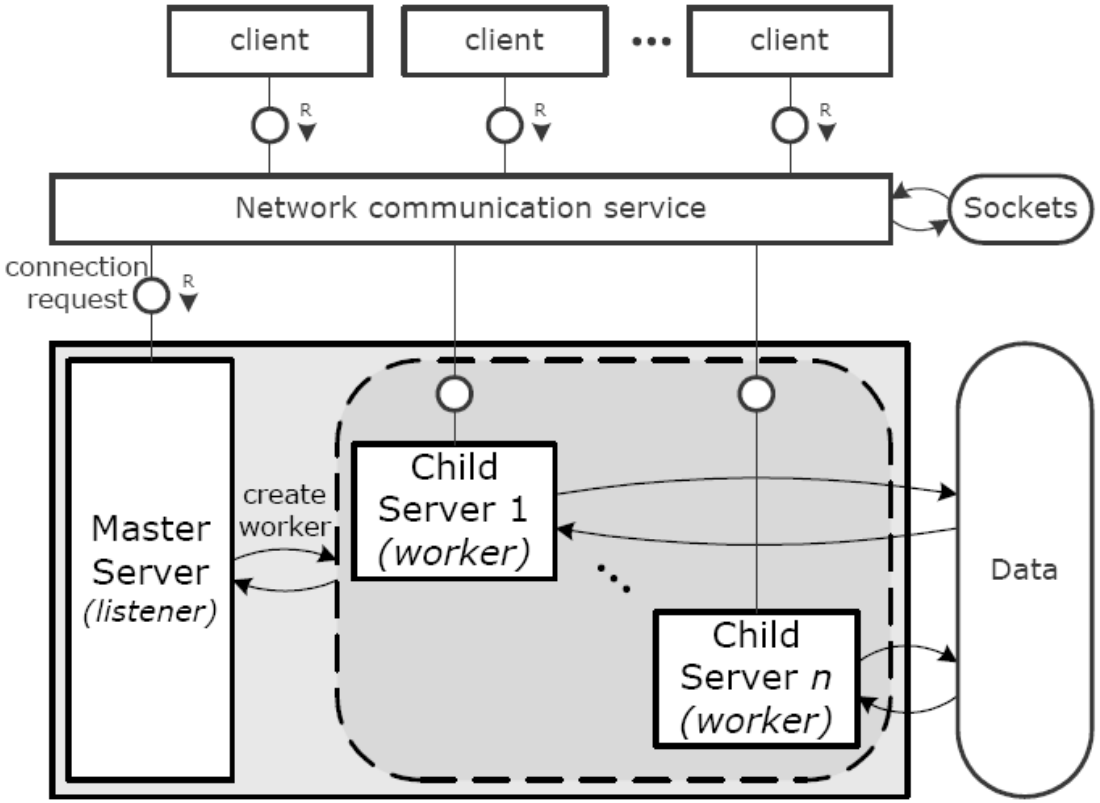
Konzeptionelle Serverpatterns – Forking Server

Dynamische Sichtweise



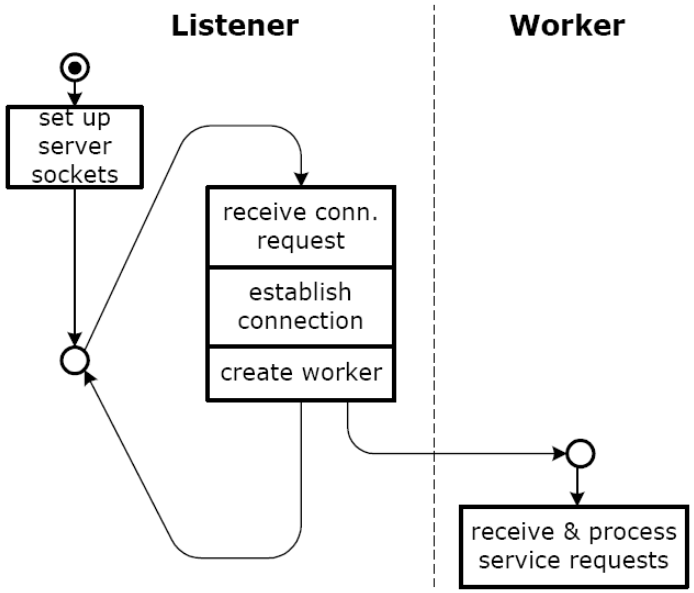
Konzeptionelle Serverpatterns – Forking Server

Statische Sichtweise



Konzeptionelle Serverpatterns – Forking Server

Dynamische Sichtweise



Konzeptionelle Serverpatterns – Forking Server

Eigenschaften:

- Die Übergabe der Client–Verbindung zwischen Listener und Worker funktioniert auch bei Verwendung von Betriebssystem–Prozessen.
- Die Anzahl der Tasks richtet sich genau nach dem aktuellen Bedarf
- Ressourcennutzung ist abhängig von der auftreten Last

Offene Fragen/Punkte

- Die Antwortzeit des Servers hängt von der Zeit ab, die das Erzeugen eines Workers erfordert
- Während ein Worker erzeugt wird können keine neuen Connect-Request bearbeitet werden
- Das ständige Erzeugen und Freigeben von Systemressourcen kostest Zeit und führt zu einer höheren Systembelastung
- Problematisch bei hoher punktueller Belastung

Konzeptionelle Serverpatterns – Worker Pool

Pattern: Worker Pool

Ausgangssituation

Man wendet für einen auftragsbearbeitenden Server das LISTENER / WORKER Pattern an.

Problem

Wie erreicht man eine möglichst kurze Antwortzeit?

Wie kann man man die Zeit für das teure Erzeugen von Workern optimieren?

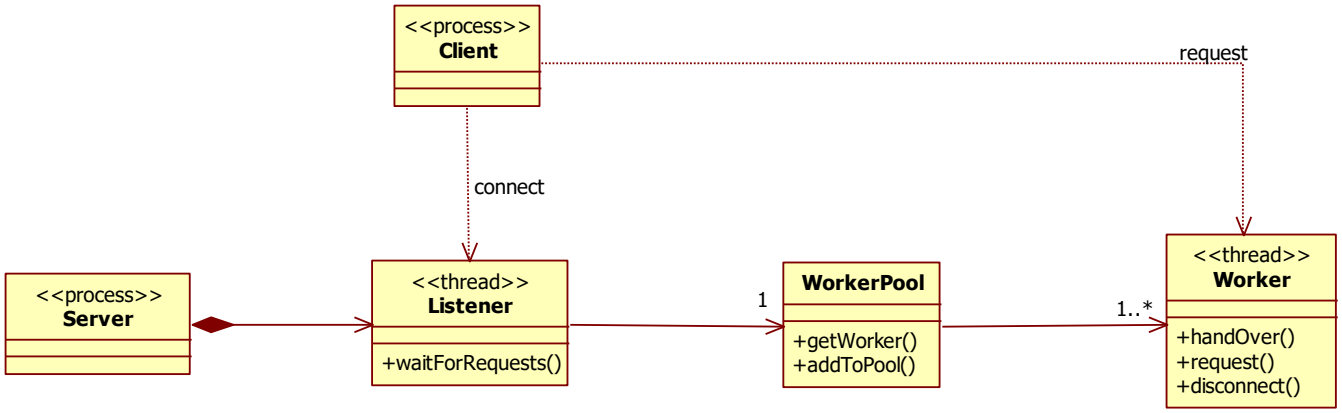
Konzeptionelle Serverpatterns – Worker Pool

Lösung:

- Erzeuge bei Start des Servers eine bestimmte Anzahl von Worker Tasks
- Solange keine Arbeit vorliegt sind diese Tasks inaktiv
- Eingehende Connect-Anforderungen werden auf einen inaktiven Task zugeteilt

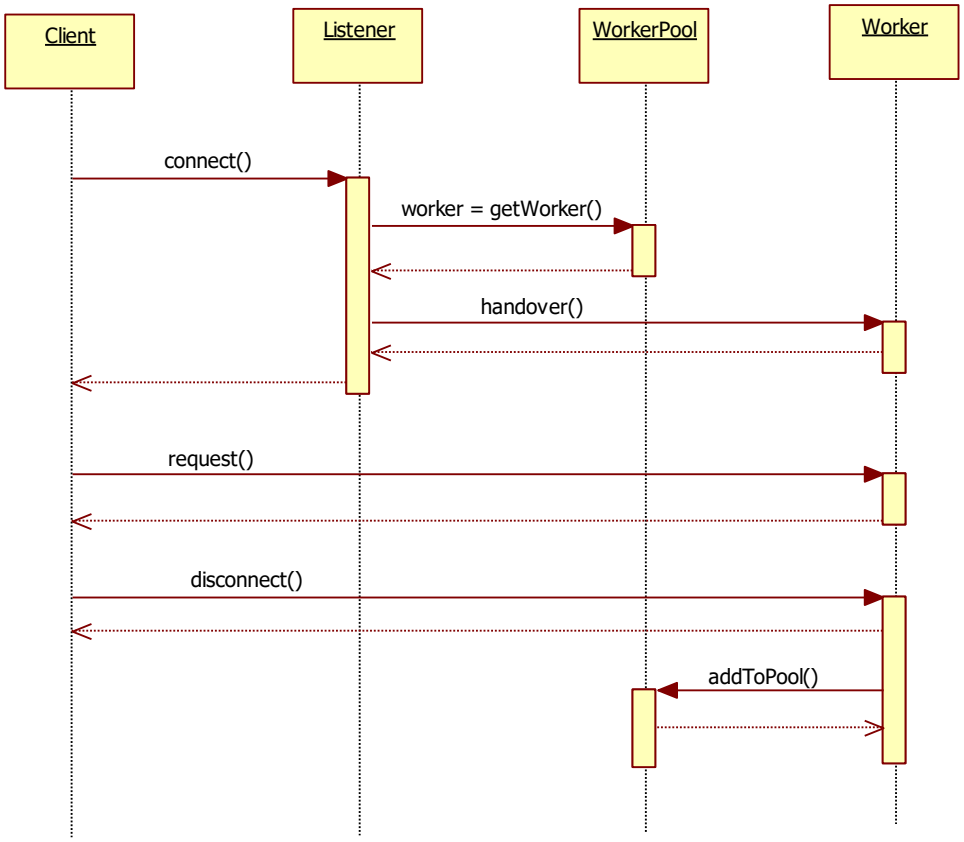
Konzeptionelle Serverpatterns – Worker Pool

Statische Sichtweise



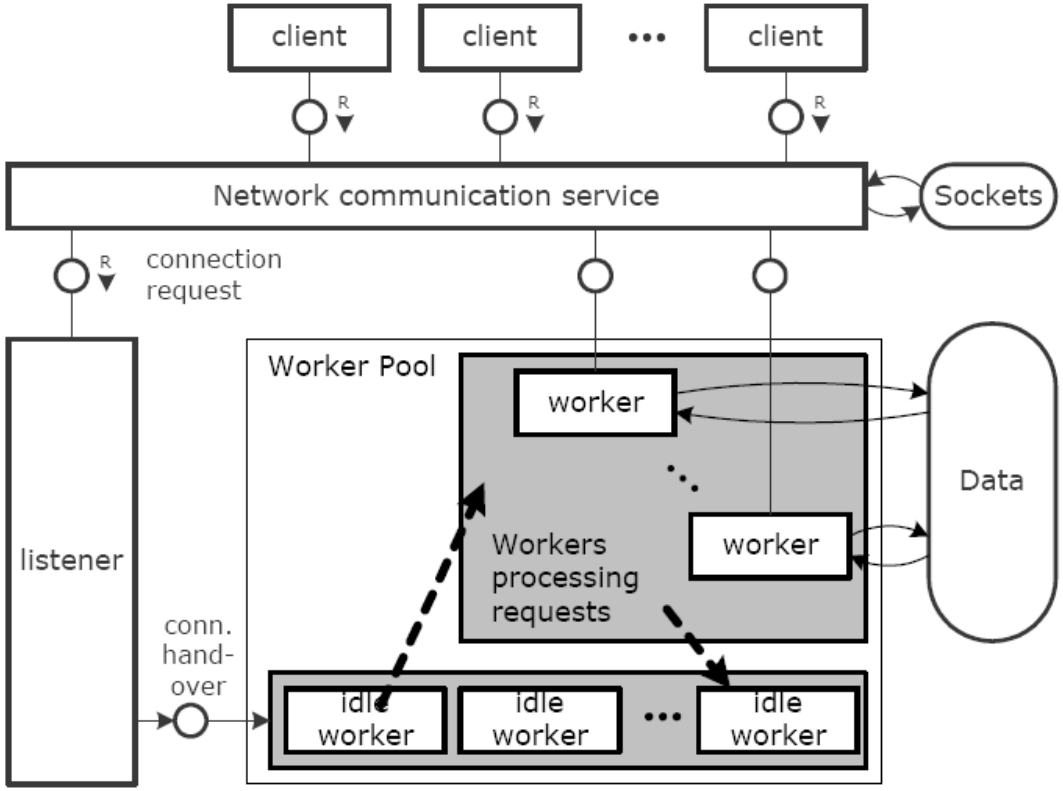
Konzeptionelle Serverpatterns – Worker Pool

Dynamische Sichtweise



Konzeptionelle Serverpatterns – Worker Pool

Statische Sichtweise



Konzeptionelle Serverpatterns – Worker Pool

Eigenschaften:

- Es geht bei der Annahme von Requests keine Zeit mehr für die Erzeugung von Worker Tasks verloren
- Die Anzahl der Worker Tasks kann begrenzt und damit für die Maschinen–Ressourcen optimiert werden
- Eingehende Anfragen müssen warten bis ein zugehöriger Worker zur Verfügung steht
- Synchronisation beim Zugriff auf Resourcen notwendig

Konzeptionelle Serverpatterns – Worker Pool

Implementierungsbeispiele:

Siehe:

- Paket fileimport.workerpool
- Paket tcpserver.workerpool

Konzeptionelle Serverpatterns – Job Queue

Pattern: Job Queue

Ausgangssituation

Man wendet das LISTENER / WORKER und das WORKER POOL Pattern an.

Problem

Wie kann die Zeit für einen Verbindungsaufbau optimiert werden, auch wenn keine freuen Worker exsistieren?

Wie übergibt der Listener eine Client–Verbindung an einen Worker

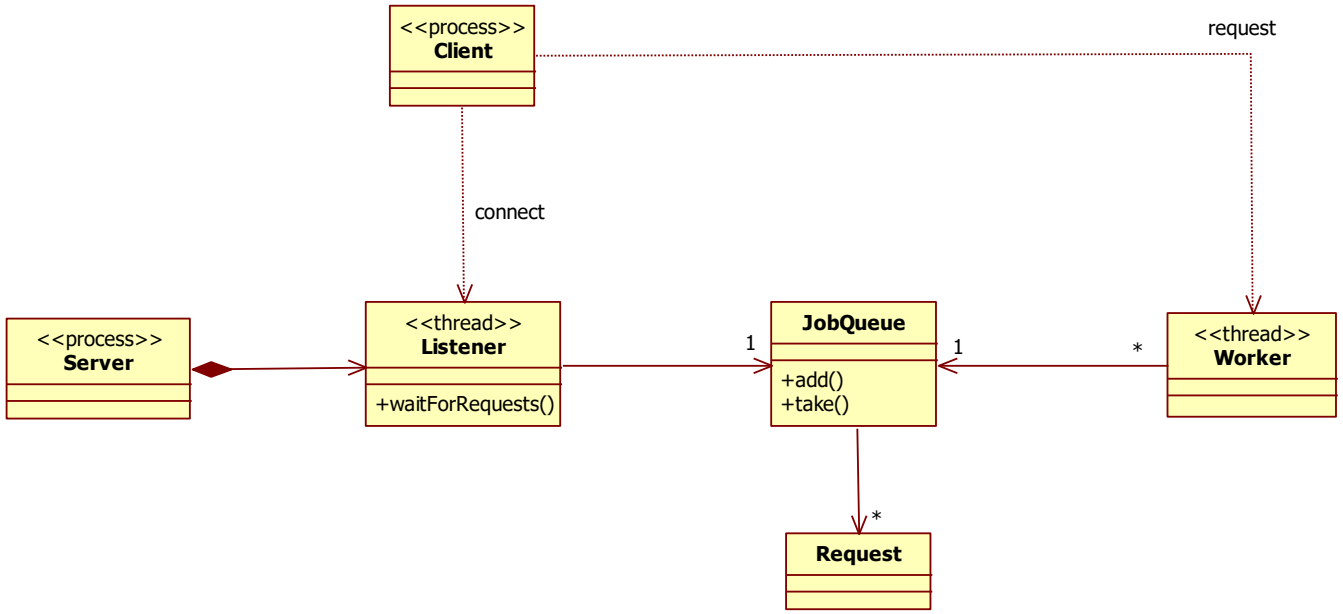
Konzeptionelle Serverpatterns – Job Queue

Lösung:

- Eine Job Queue dient zur Übergabe der Client–Verbindung zwischen Listener und Worker.
- Alle inaktiven Worker warten darauf, dass ein Job in die Queue kommt.
- Nach Annahme eines Connection Requests schiebt der Listener die Client–Verbindung als Job in die Queue und weckt damit den ersten inaktiven Worker
- Der Worker nimmt den Job aus der Queue und kommuniziert mit dem Client
- Falls keine inaktiven Worker Tasks warten, füllt sich die Queue mit jedem Connection Request.

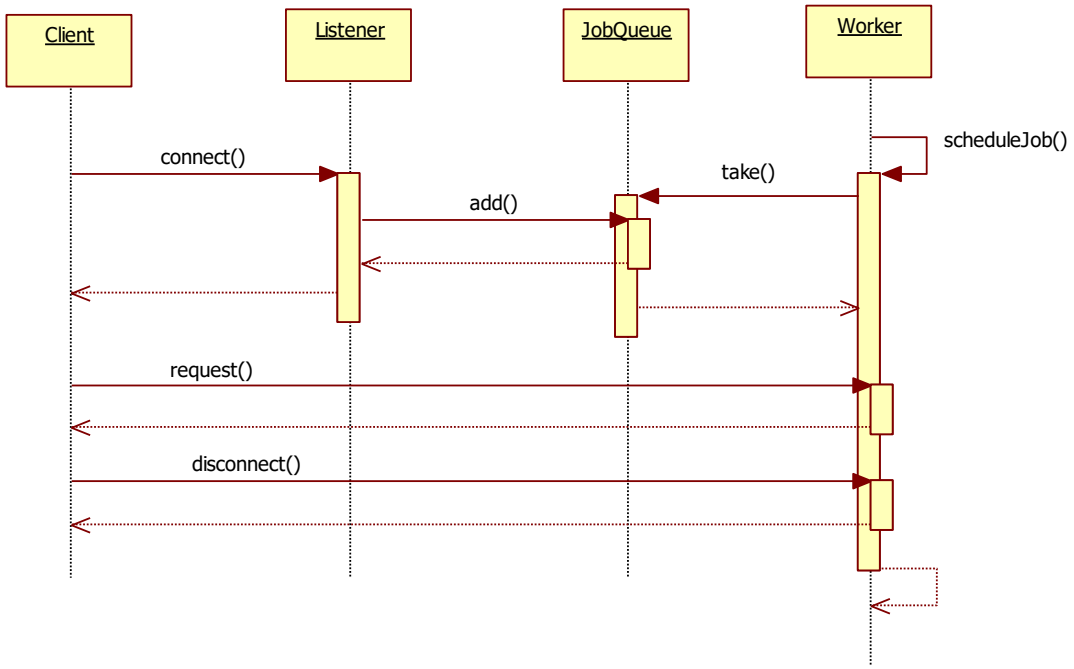
Konzeptionelle Serverpatterns – Job Queue

Statische Sichtweise



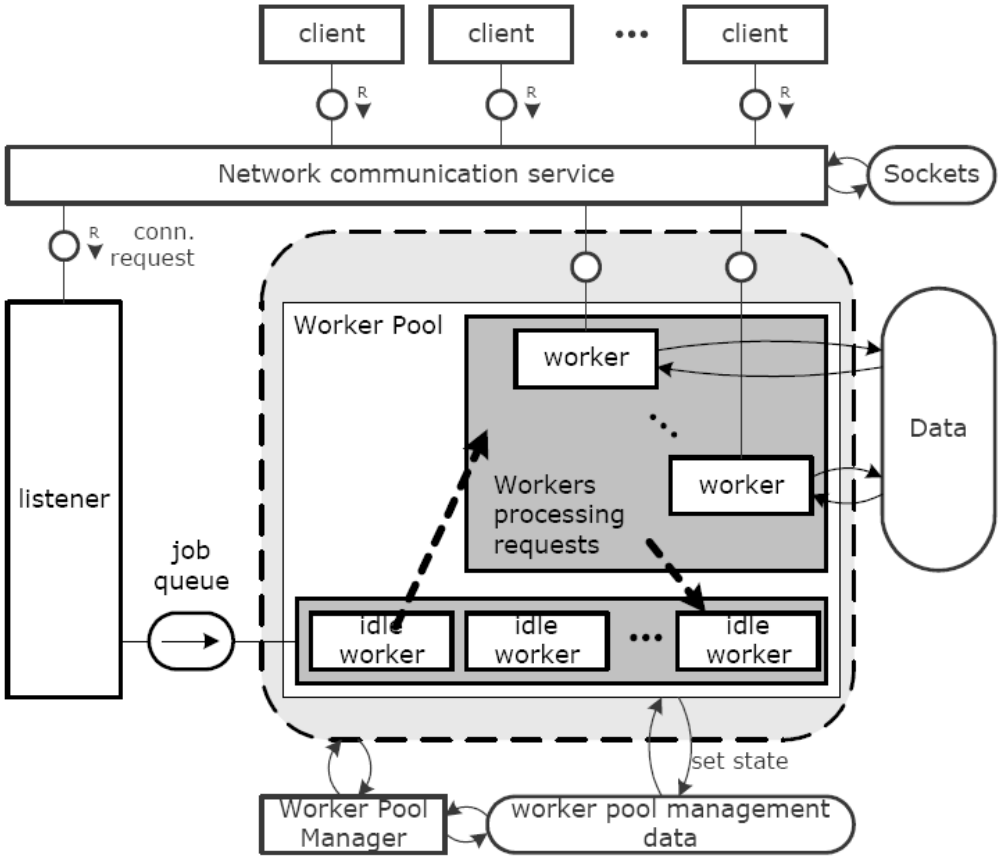
Konzeptionelle Serverpatterns – Job Queue

Dynamische Sichtweise



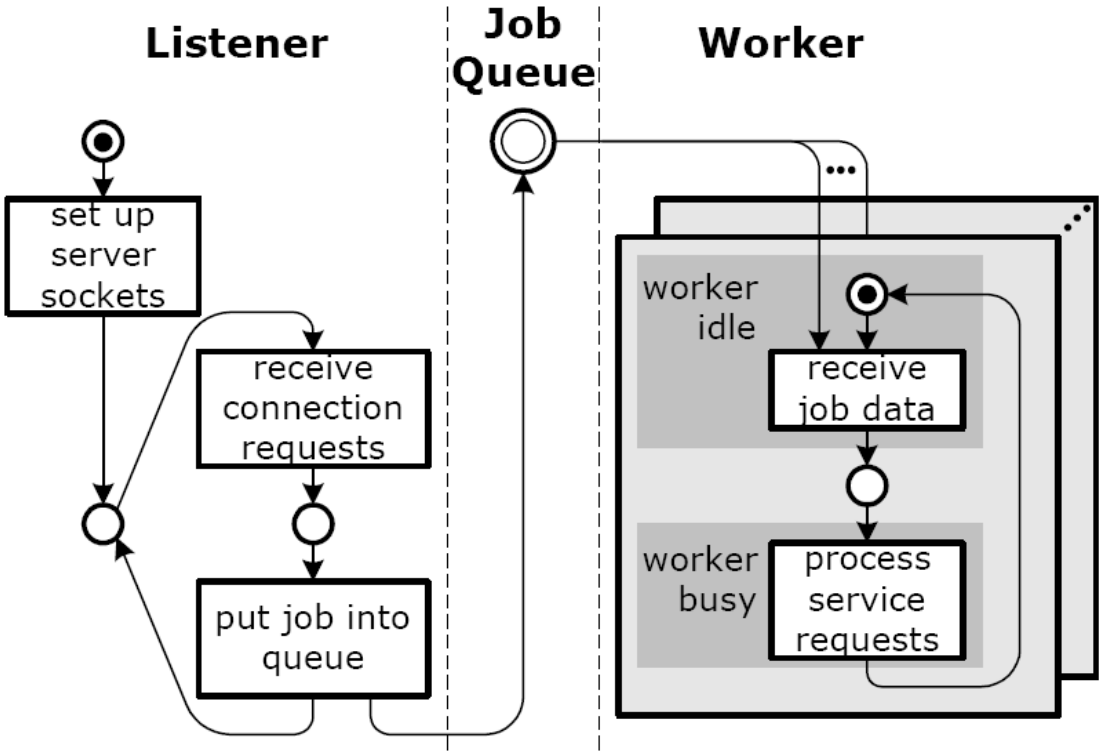
Konzeptionelle Serverpatterns – Job Queue

Statische Sichtweise



Konzeptionelle Serverpatterns – Job Queue

Dynamische Sichtweise



Konzeptionelle Serverpatterns – Job Queue

Eigenschaften:

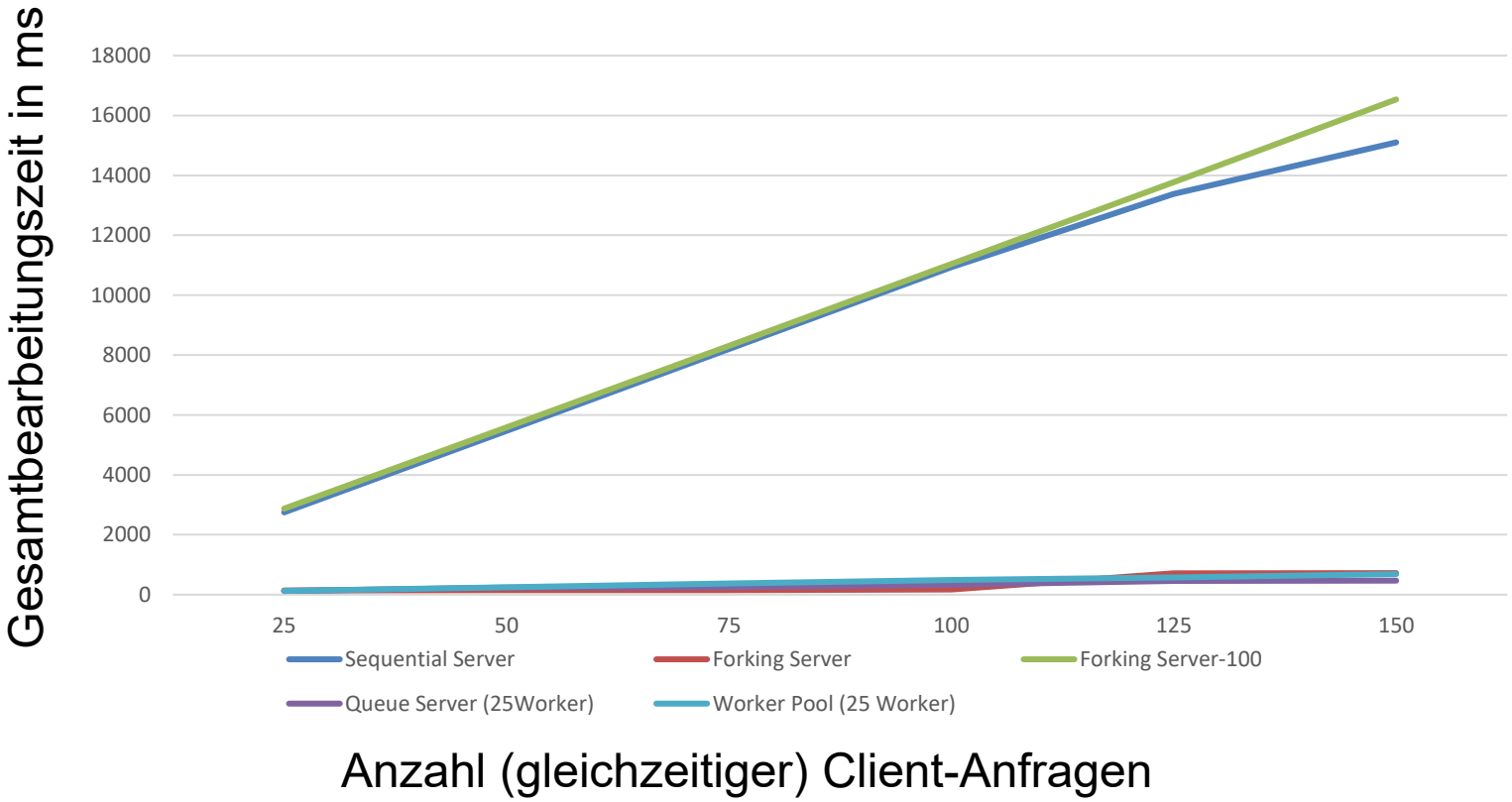
- Der Listener nimmt mit der kürzest möglichen Antwortzeit Connection Requests an
- Durch die Job Queue sind auch Worker Pools mit statischer Anzahl an Workern effizient
- ein Client muss eventuell etwas warten, bis sein Service Request angenommen und bearbeitet wird
- Falls möglich kann die Queue auch persistent abgelegt werden

Offene Fragen/Punkte

- Wie kann die Auslastung der Queue begrenzt werden?

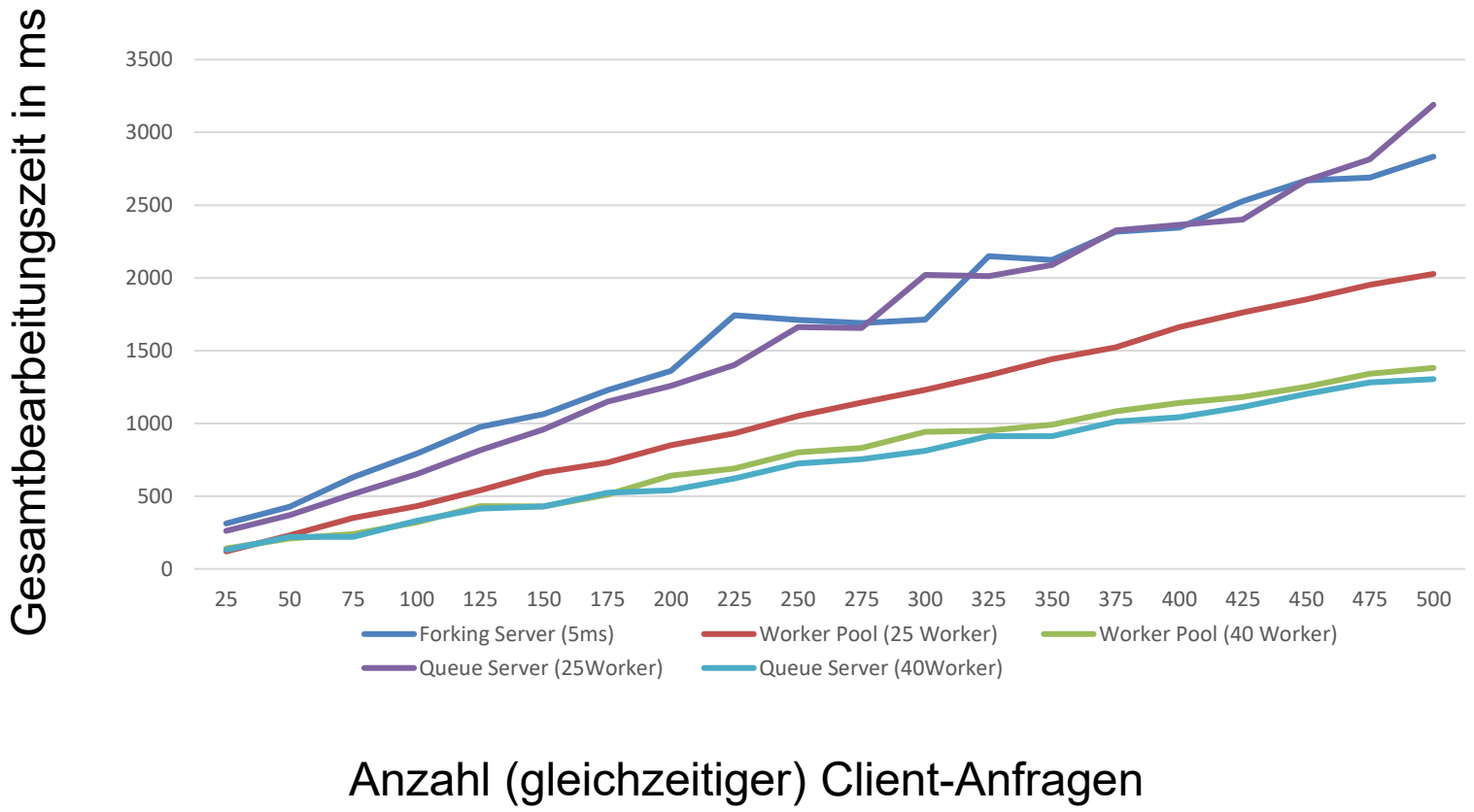
Konzeptionelle Serverpatterns – Vergleich

Performancevergleich der vorgestellten Patterns



Konzeptionelle Serverpatterns – Vergleich

Performancevergleich der vorgestellten Patterns



Prozess-/Ablaufsicht

Fazit

- Die vorgestellten Pattern erlauben die bewusste Steuerung der nicht-funktionalen Anforderungen
- Für jedes (Server-)basierte System ist die Verarbeitung von Aufträgen von zentraler Bedeutung.
- Eine definierte Resourcen-Auslastung ist Garant für eine stabiles und zuverlässiges System welches in definierten Rahmenparameter arbeitet
- Verschiedene Pattern können in einer Applikation kombiniert werden um das optimale Ablaufverhalten zu erzielen
- Die Element der Ablaufsicht können als Komponenten an die restliche Applikation exportiert werden